

Loading and Reading the Data

```
In [ ]: import pandas as pd
import warnings
warnings.filterwarnings('ignore')

# Load the dataset
data = pd.read_excel(r'C:\Users\moses_y\OneDrive\Desktop\ML Projects\Pipeline\Quote Pipeline.xlsx', header=1)
```

Columns in the Dataset

```
In [ ]: # Strip leading/trailing spaces from column names
data.columns = data.columns.str.strip()

# Print column names to verify
print(data.columns)

Index(['CustomerNo', 'Customer', 'Market Channel', 'Description',
       'Div Mfr Rep Name', 'Internal Account Manager', 'Salesperson',
       'Sell-To Contact', 'Ship To Name', 'Sell-To City', 'Sell-To State',
       'Sell-To Zip', 'Sell-to Phone No', 'Quote Type', 'Quote #',
       'Quote Date', 'Primary', 'Item', 'QuotePipeLine.Description',
       'Perceived Quantity', 'Total Value', 'Stage', 'Probability Value',
       'LeadSource', 'Follow-Up Needed', 'Follow-Up Date', 'Document Date',
       'Expiration Date', 'Price Group', 'No of Archived Versions',
       'Quote Month', 'Document Month', 'Project Name', 'Artwork Path'],
      dtype='object')
```

First Few Rows of The Data

```
In [ ]: data.head()
```

```
Out [ ]:
```

	CustomerNo	Customer	Market Channel	Description	Div Mfr Rep Name	Internal Account Manager	Salesperson	Sell-To Contact	Ship To Name	Sell-To City	...	Follow-Up Needed	Follow-Up Date
0	D011358	1st Class Solutions	7	Contract Furniture	Joan Kiser	Andrew Karpuk	CFDALLAS	Kim Newsom		LUBBOCK	...	NaN	NaN
1	D011358	1st Class Solutions	7	Contract Furniture	Joan Kiser	Andrew Karpuk	CFDALLAS	Kim Newsom		LUBBOCK	...	NaN	NaN
2	D011358	1st Class Solutions	7	Contract Furniture	Joan Kiser	Andrew Karpuk	CFDALLAS	Kim Newsom		LUBBOCK	...	NaN	NaN
3	D009405	2010 Office Furniture	7	Contract Furniture	Bismark Padilla	NaN	CFSTHWEST	Irene Almazar	2010 Office Furniture	South El Monte	...	NaN	NaN
4	D009405	2010 Office Furniture	7	Contract Furniture	Bismark Padilla	NaN	CFSTHWEST	Irene Almazar	2010 Office Furniture	South El Monte	...	NaN	NaN

5 rows × 34 columns

Number of rows and columns

```
In [ ]: print("Number of rows and columns in the dataset:", data.shape)

Number of rows and columns in the dataset: (31390, 34)
```

Missing values in each column:

```
In [ ]: print("Missing values in each column:\n", data.isnull().sum())
```

```

Missing values in each column:
  CustomerNo          4
  Customer            4
  Market Channel      13
  Description          13
  Div Mfr Rep Name    4602
  Internal Account Manager 6788
  Salesperson         5
  Sell-To Contact     4899
  Ship To Name        4
  Sell-To City        252
  Sell-To State       307
  Sell-To Zip         335
  Sell-to Phone No    440
  Quote Type         53
  Quote #            4
  Quote Date         4
  Primary            4
  Item              121
  QuotePipeline.Description 63
  Perceived Quantity  4
  Total Value        4
  Stage             56
  Probability Value   3
  LeadSource         24440
  Follow-Up Needed   26956
  Follow-Up Date     31390
  Document Date      4
  Expiration Date    560
  Price Group        4
  No of Archived Versions 4
  Quote Month        4
  Document Month     4
  Project Name       8330
  Artwork Path       31143
dtype: int64

```

Training a Random Forest Model for Analysis.

1. Reading Columns.
2. Finding Out Missing values in each column
3. Removing Missing values from each column
4. Finding Out the number of unique values in each column
5. Fixing Imbalanced `Stage` column in the dataset.
6. Training the Model
7. Model accuracy `97.6%` accurate
8. `Top 10` features in our model

```

In [ ]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
from sklearn.feature_selection import SelectKBest, chi2
from imblearn.over_sampling import SMOTE
from scipy import sparse
import matplotlib.pyplot as plt
import seaborn as sns

# Loading the data
data = pd.read_excel(r'C:\Users\moses_y\OneDrive\Desktop\ML Projects\Pipeline\Quote Pipeline.xlsx', header=1)

# Print column names to verify
print("Columns in the dataset:", data.columns)

# Strip leading/trailing spaces from column names
data.columns = data.columns.str.strip()

# Check if 'Stage' is in the columns
if 'Stage' not in data.columns:
    raise KeyError("The 'Stage' column is not found in the dataset. Please check the column names.")

# Handling Missing Values

```

```

missing_values = data.isnull().sum()
missing_percentage = (missing_values / len(data)) * 100
print(missing_percentage)

# Drop columns with more than 25% missing values
threshold = 25
data_dropped = data.loc[:, missing_percentage < threshold]
print(f"Columns after dropping those with >{threshold}% missing values:", data_dropped.columns)

# Drop specified columns
data_dropped = data_dropped.drop(columns=['Document Month', 'Market Channel'])

# Identify categorical and numerical columns
categorical_features = data_dropped.select_dtypes(include=['object']).columns
numerical_features = data_dropped.select_dtypes(include=[np.number]).columns

# Remove 'Stage' from categorical_features if present
if 'Stage' in categorical_features:
    categorical_features = categorical_features.drop('Stage')

# Impute missing values
categorical_imputer = SimpleImputer(strategy='constant', fill_value='Unknown')
data_dropped.loc[:, categorical_features] = categorical_imputer.fit_transform(data_dropped.loc[:, categorical_features])

numerical_imputer = SimpleImputer(strategy='most_frequent')
data_dropped.loc[:, numerical_features] = numerical_imputer.fit_transform(data_dropped.loc[:, numerical_features])

# Map 'Stage' column to combined classes
stage_mapping = {
    'lost': 0,
    'sale': 1,
    'open': 2,
    'open - new': 2,
    'open - active review': 2,
    'open - gained aprvl': 2,
    'unknown': 3
}
data_dropped['Stage'] = data_dropped['Stage'].str.lower().map(stage_mapping)

```

```

# Drop rows with missing 'Stage' values
data_dropped.dropna(subset=['Stage'], inplace=True)

# Encode Categorical Variables with Sparse Matrix
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=True)
encoded_features = encoder.fit_transform(data_dropped[categorical_features])

# Combine numerical and encoded categorical features
numerical_df = pd.DataFrame(data_dropped[numerical_features], columns=numerical_features)
data_final_sparse = sparse.hstack([sparse.csr_matrix(numerical_df.values), encoded_features])

# Print the distribution of the target variable
print("Distribution of target variable 'Stage':")
print(data_dropped['Stage'].value_counts())

# Splitting the data into training and testing sets
X = data_final_sparse
y = data_dropped['Stage']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Applying SMOTE to balance the classes
smote = SMOTE(random_state=42)
X_train_balanced, y_train_balanced = smote.fit_resample(X_train, y_train)

# Model Building: Random Forest Classifier for multi-class classification
model = RandomForestClassifier(random_state=42)
model.fit(X_train_balanced, y_train_balanced)

# Model Evaluation
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
print('Accuracy:', accuracy_score(y_test, y_pred))

# Feature Importance
# Use SelectKBest to reduce the number of features for feature importance plotting
selector = SelectKBest(chi2, k=15).fit(X_train_balanced, y_train_balanced)
selected_features = selector.get_support(indices=True)

# Extract the feature importances for the selected features only

```

```

selected_feature_importances = model.feature_importances_[selected_features]

# Get feature names
encoded_feature_names = encoder.get_feature_names_out(categorical_features)
numerical_feature_names = numerical_features.tolist()
all_feature_names = numerical_feature_names + encoded_feature_names.tolist()
selected_feature_names = [all_feature_names[i] for i in selected_features]

# Plot feature importances for the selected features
plt.figure(figsize=(12, 8))
feature_importances = pd.Series(selected_feature_importances, index=selected_feature_names)
feature_importances.nlargest(10).plot(kind='barh')
plt.title('Top 10 Feature Importances')
plt.xlabel('Importance')
plt.ylabel('Features')
plt.grid(True)
plt.show()

```

Columns in the dataset: Index(['CustomerNo', 'Customer', 'Market Channel', 'Description', 'Div Mfr Rep Name', 'Internal Account Manager', 'Salesperson', 'Sell-To Contact', 'Ship To Name', 'Sell-To City', 'Sell-To State', 'Sell-To Zip', 'Sell-to Phone No', 'Quote Type', 'Quote #', 'Quote Date', 'Primary', 'Item', 'QuotePipeline.Description', 'Perceived Quantity', 'Total Value', 'Stage', 'Probability Value', 'LeadSource', 'Follow-Up Needed', 'Follow-Up Date', 'Document Date', 'Expiration Date', 'Price Group', 'No of Archived Versions', 'Quote Month', 'Document Month', 'Project Name', 'Artwork Path'], dtype='object')

CustomerNo	0.012743
Customer	0.012743
Market Channel	0.041414
Description	0.041414
Div Mfr Rep Name	14.660720
Internal Account Manager	21.624721
Salesperson	0.015929
Sell-To Contact	15.606881
Ship To Name	0.012743
Sell-To City	0.802803
Sell-To State	0.978018
Sell-To Zip	1.067219
Sell-to Phone No	1.401720
Quote Type	0.168844
Quote #	0.012743
Quote Date	0.012743
Primary	0.012743
Item	0.385473
QuotePipeline.Description	0.200701
Perceived Quantity	0.012743
Total Value	0.012743
Stage	0.178401
Probability Value	0.009557
LeadSource	77.859191
Follow-Up Needed	85.874482
Follow-Up Date	100.000000
Document Date	0.012743
Expiration Date	1.784008
Price Group	0.012743

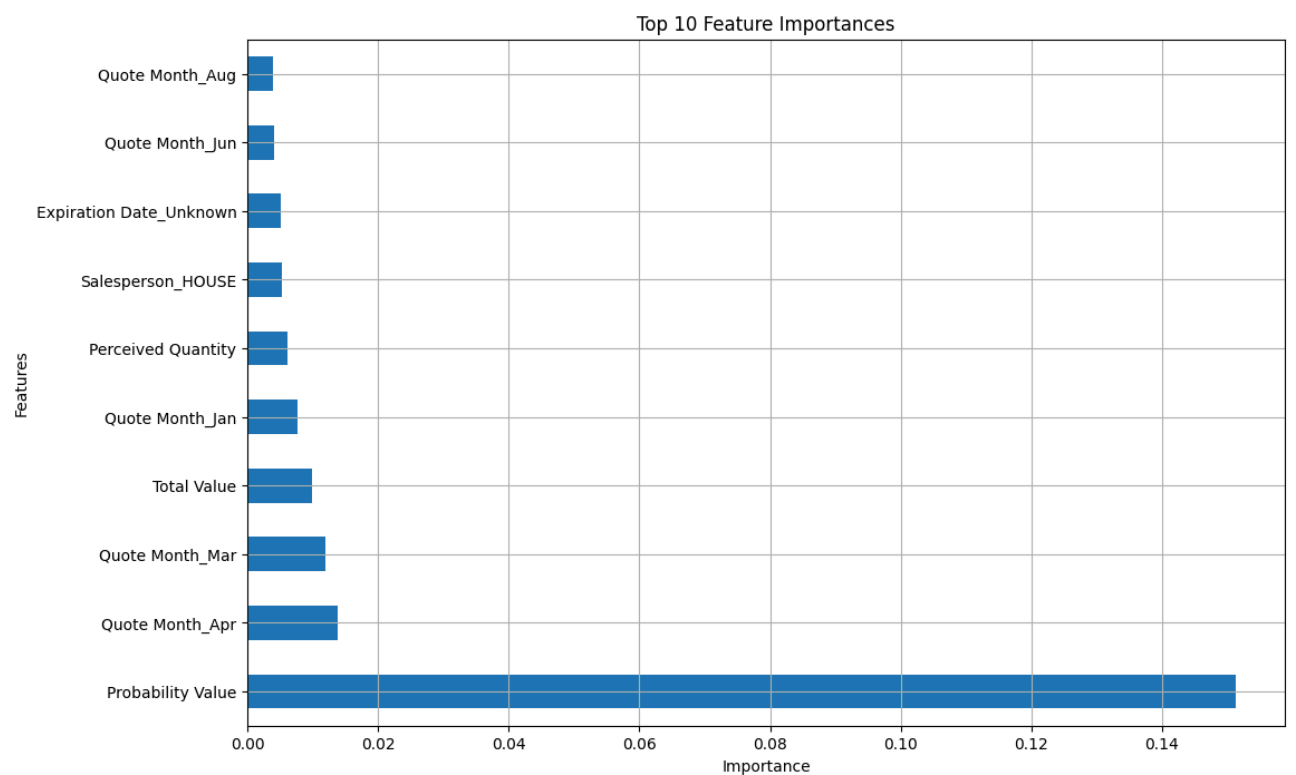
```

No of Archived Versions      0.012743
Quote Month                  0.012743
Document Month               0.012743
Project Name                 26.537114
Artwork Path                 99.213125
dtype: float64
Columns after dropping those with >25% missing values: Index(['CustomerNo', 'Customer', 'Market Channel', 'Description',
'Div Mfr Rep Name', 'Internal Account Manager', 'Salesperson',
'Sell-To Contact', 'Ship To Name', 'Sell-To City', 'Sell-To State',
'Sell-To Zip', 'Sell-to Phone No', 'Quote Type', 'Quote #',
'Quote Date', 'Primary', 'Item', 'QuotePipeLine.Description',
'Perceived Quantity', 'Total Value', 'Stage', 'Probability Value',
'Document Date', 'Expiration Date', 'Price Group',
'No of Archived Versions', 'Quote Month', 'Document Month'],
dtype='object')
Distribution of target variable 'Stage':
Stage
0.0    16765
1.0     7633
2.0     6936
Name: count, dtype: int64

```

	precision	recall	f1-score	support
0.0	0.98	1.00	0.99	3353
1.0	0.98	0.94	0.96	1527
2.0	0.96	0.96	0.96	1387
accuracy			0.98	6267
macro avg	0.97	0.97	0.97	6267
weighted avg	0.98	0.98	0.98	6267

Accuracy: 0.9763842348811234



Detailed Quantity Analysis of Factors Contributing to Winning and Losing Quotes

1. Products (Items)

- **Question:** Which products are associated with higher win rates and which are linked to losses?

- **Analysis:**

- **Insight:** Identify top 10 products with the highest and lowest win rates. Adjust marketing and sales strategies accordingly.
- **Top 10 Items** These items have a perfect win rate, indicating that they are very successful. The company should ensure these products are well-promoted and stocked to maximize opportunities.

```
In [ ]: # Analysis and sorting
def analyze_item_impact(data, feature_name='Item', target_name='Stage'):
    item_success = data.groupby([feature_name, target_name]).size().unstack().fillna(0)
    item_success['Win_Quantity'] = item_success[1.0]
    item_success['Total_Quantity'] = item_success[1.0] + item_success[0.0]
    item_success['Win_Rate'] = item_success['Win_Quantity'] / item_success['Total_Quantity']
    return item_success.sort_values(by=['Win_Quantity', 'Win_Rate'], ascending=[False, False])

item_impact = analyze_item_impact(data_dropped)
print("Top 10 Items by Win Quantity and Win Rate:")
print(item_impact.head(10)) # Top 10 items sorted by win quantity and win rate
```

Top 10 Items by Win Quantity and Win Rate:

Stage	0.0	1.0	2.0	Win_Quantity	Total_Quantity	Win_Rate
Item						
ARIASM46WH	655.0	368.0	293.0	368.0	1023.0	0.359726
ARIASM48WH	507.0	296.0	199.0	296.0	803.0	0.368618
M1-48-4	411.0	210.0	158.0	210.0	621.0	0.338164
M1-46-4	415.0	201.0	156.0	201.0	616.0	0.326299
AMMT12	351.0	193.0	194.0	193.0	544.0	0.354779
SPMD-M1	398.0	182.0	160.0	182.0	580.0	0.313793
SPMD-ARIA	484.0	161.0	147.0	161.0	645.0	0.249612
ARIASM34WH	307.0	159.0	158.0	159.0	466.0	0.341202
M1-34-1	231.0	123.0	90.0	123.0	354.0	0.347458
ARIASM45WH	184.0	110.0	78.0	110.0	294.0	0.374150

Insight: These items have not been won yet. The company should investigate why these items are not successful and consider either discontinuing them or changing the sales strategy for these products.

```
In [ ]: print("Bottom 10 Items by Win Quantity and Win Rate:")
print(item_impact.tail(10)) # Bottom 10 items sorted by win quantity and win rate
```

Bottom 10 Items by Win Quantity and Win Rate:

Stage	0.0	1.0	2.0	Win_Quantity	Total_Quantity	Win_Rate
Item						
SPMD-M1-MULTI	0.0	0.0	14.0	0.0	0.0	NaN
SPMD-PRS4M	0.0	0.0	1.0	0.0	0.0	NaN
SPMD-SARIA-A	0.0	0.0	5.0	0.0	0.0	NaN
TF18-93	0.0	0.0	1.0	0.0	0.0	NaN
TF23-95	0.0	0.0	1.0	0.0	0.0	NaN
TF23-96	0.0	0.0	1.0	0.0	0.0	NaN
TF34-98	0.0	0.0	1.0	0.0	0.0	NaN
WRMM46	0.0	0.0	1.0	0.0	0.0	NaN
WSD2-5747-C	0.0	0.0	3.0	0.0	0.0	NaN
WTR44-CF	0.0	0.0	1.0	0.0	0.0	NaN

2. Customers

- **Question:** Which customers are associated with higher success rates?

- **Analysis:**

- **Insight:** These customers have a perfect win rate. The company should focus on maintaining and strengthening relationships with these clients.

```
In [ ]: def analyze_customer_impact(data, feature_name='Customer', target_name='Stage'):
    customer_success = data.groupby([feature_name, target_name]).size().unstack().fillna(0)
    customer_success['Win_Quantity'] = customer_success[1.0]
    customer_success['Total_Quantity'] = customer_success[1.0] + customer_success[0.0]
    customer_success['Win_Rate'] = customer_success['Win_Quantity'] / customer_success['Total_Quantity']
    return customer_success.sort_values(by=['Win_Quantity', 'Win_Rate'], ascending=[False, False])

customer_impact = analyze_customer_impact(data_dropped)
print("Top 10 Customers by Win Quantity and Win Rate:")
print(customer_impact.head(10)) # Top 10 customers sorted by win quantity and win rate
```

Top 10 Customers by Win Quantity and Win Rate:

Stage	0.0	1.0	2.0	Win_Quantity \
Customer				
Hertz Furniture Systems Corp	270.0	165.0	87.0	165.0
Staples Business Interiors	189.0	148.0	113.0	148.0
Interior Environments	69.0	126.0	20.0	126.0
Product Builder	818.0	126.0	303.0	126.0
Interior Investments	26.0	117.0	5.0	117.0
Government -GSA	103.0	117.0	58.0	117.0
GMI Companies - EndUser	1054.0	108.0	679.0	108.0
Price Modern	86.0	98.0	40.0	98.0
WorkSpace Interiors by OD-Veyer/ODP	302.0	97.0	106.0	97.0
RCF Group	133.0	82.0	56.0	82.0

Stage	Total_Quantity	Win_Rate
Customer		
Hertz Furniture Systems Corp	435.0	0.379310
Staples Business Interiors	337.0	0.439169
Interior Environments	195.0	0.646154
Product Builder	944.0	0.133475
Interior Investments	143.0	0.818182
Government -GSA	220.0	0.531818
GMI Companies - EndUser	1162.0	0.092943
Price Modern	184.0	0.532609
WorkSpace Interiors by OD-Veyer/ODP	399.0	0.243108
RCF Group	215.0	0.381395

- **Insight:** These customers have not yet resulted in any wins. It may be necessary to revisit the sales approach or consider if these are viable targets.

```
In [ ]: print("Bottom 10 Customers by Win Quantity and Win Rate:")
        print(customer_impact.tail(10)) # Bottom 10 customers sorted by win quantity and win rate
```

Bottom 10 Customers by Win Quantity and Win Rate:

Stage	0.0	1.0	2.0	Win_Quantity \
Customer				
Washington Door and Hardware LLC	0.0	0.0	7.0	0.0
Wasserstrom Company	0.0	0.0	2.0	0.0
Watson's Office City	0.0	0.0	1.0	0.0
Wilson Bauhaus Interiors-TULSA	0.0	0.0	8.0	0.0
Wilton's Office Works-Bryan	0.0	0.0	2.0	0.0
Witt's Everything for the Ofc	0.0	0.0	2.0	0.0
Workplace Solutions Inc-Dallas	0.0	0.0	10.0	0.0
Workscapes Inc-Jacksonville	0.0	0.0	2.0	0.0
Workspace Innovations-Fort Collins	0.0	0.0	2.0	0.0
Workspace Solutions PA, Inc.	0.0	0.0	2.0	0.0

Stage	Total_Quantity	Win_Rate
Customer		
Washington Door and Hardware LLC	0.0	NaN
Wasserstrom Company	0.0	NaN
Watson's Office City	0.0	NaN
Wilson Bauhaus Interiors-TULSA	0.0	NaN
Wilton's Office Works-Bryan	0.0	NaN
Witt's Everything for the Ofc	0.0	NaN
Workplace Solutions Inc-Dallas	0.0	NaN
Workscapes Inc-Jacksonville	0.0	NaN
Workspace Innovations-Fort Collins	0.0	NaN
Workspace Solutions PA, Inc.	0.0	NaN

3. Rep Groups (Div Mfr Rep Name)

- **Question:** Which representative groups contribute most to winning or losing quotes?
- **Analysis:**
 - **Insight:** These rep groups have a perfect win rate. Best practices from these reps should be analyzed and possibly adopted by other reps.

```
In [ ]: def analyze_rep_group_impact(data, feature_name='Div Mfr Rep Name', target_name='Stage'):
        rep_group_success = data.groupby([feature_name, target_name]).size().unstack().fillna(0)
```

```

rep_group_success['Win_Quantity'] = rep_group_success[1.0]
rep_group_success['Total_Quantity'] = rep_group_success[1.0] + rep_group_success[0.0]
rep_group_success['Win_Rate'] = rep_group_success['Win_Quantity'] / rep_group_success['Total_Quantity']
return rep_group_success.sort_values(by=['Win_Quantity', 'Win_Rate'], ascending=[False, False])

rep_group_impact = analyze_rep_group_impact(data_dropped)
print("Top 10 Rep Groups by Win Quantity and Win Rate:")
print(rep_group_impact.head(10)) # Top 10 rep groups sorted by win quantity and win rate

```

Top 10 Rep Groups by Win Quantity and Win Rate:

Stage	0.0	1.0	2.0	Win_Quantity	Total_Quantity	\
Div Mfr Rep Name						
Customer Service	2202.0	742.0	960.0	742.0	2944.0	
Unknown	2801.0	475.0	1321.0	475.0	3276.0	
Michael Isaac	668.0	406.0	224.0	406.0	1074.0	
Dan Binford	790.0	399.0	319.0	399.0	1189.0	
Ken Gibson	566.0	378.0	181.0	378.0	944.0	
Monique Sluzynski	555.0	318.0	204.0	318.0	873.0	
Lynn Headley	416.0	277.0	179.0	277.0	693.0	
Rick Allor	247.0	227.0	108.0	227.0	474.0	
Rich Sassone	448.0	216.0	126.0	216.0	664.0	
Peter Telegadis	257.0	203.0	118.0	203.0	460.0	

Stage	Win_Rate
Div Mfr Rep Name	
Customer Service	0.252038
Unknown	0.144994
Michael Isaac	0.378026
Dan Binford	0.335576
Ken Gibson	0.400424
Monique Sluzynski	0.364261
Lynn Headley	0.399711
Rick Allor	0.478903
Rich Sassone	0.325301
Peter Telegadis	0.441304

- **Insight:** These reps have not won any quotes. Additional training or strategy adjustments may be needed.

```

In [ ]: print("Bottom 10 Rep Groups by Win Quantity and Win Rate:")
print(rep_group_impact.tail(10)) # Bottom 10 rep groups sorted by win quantity and win rate

```

Bottom 10 Rep Groups by Win Quantity and Win Rate:

Stage	0.0	1.0	2.0	Win_Quantity	Total_Quantity	Win_Rate
Div Mfr Rep Name						
Jan Rawlins	1.0	0.0	0.0	0.0	1.0	0.0
Jim Harter	2.0	0.0	0.0	0.0	2.0	0.0
Kim Orth	1.0	0.0	0.0	0.0	1.0	0.0
Migdalia Ramos	1.0	0.0	0.0	0.0	1.0	0.0
Rick Kaufman	1.0	0.0	0.0	0.0	1.0	0.0
Stuart Robertson	12.0	0.0	0.0	0.0	12.0	0.0
xJoe Victoriano	1.0	0.0	0.0	0.0	1.0	0.0
Chelsea Reeves	0.0	0.0	1.0	0.0	0.0	NaN
Christina Benson	0.0	0.0	1.0	0.0	0.0	NaN
Steve Caruso	0.0	0.0	5.0	0.0	0.0	NaN

4. Salespeople

- **Question:** Which salespeople have the highest success rates?
- **Analysis:**
 - **Insight:** The salespeople listed have high win rates. Their techniques and strategies should be analyzed for best practices.

```

In [ ]: def analyze_salesperson_impact(data, feature_name='Salesperson', target_name='Stage'):
    salesperson_success = data.groupby([feature_name, target_name]).size().unstack().fillna(0)
    salesperson_success['Win_Quantity'] = salesperson_success[1.0]
    salesperson_success['Total_Quantity'] = salesperson_success[1.0] + salesperson_success[0.0]
    salesperson_success['Win_Rate'] = salesperson_success['Win_Quantity'] / salesperson_success['Total_Quantity']
    return salesperson_success.sort_values(by=['Win_Quantity', 'Win_Rate'], ascending=[False, False])

salesperson_impact = analyze_salesperson_impact(data_dropped)
print("Top 10 Salespeople by Win Quantity and Win Rate:")
print(salesperson_impact.head(10)) # Top 10 salespeople sorted by win quantity and win rate

```


Top 10 Salespeople by Win Quantity and Win Rate:

Stage	0.0	1.0	2.0	Win_Quantity	Total_Quantity	Win_Rate
Salesperson						
HG	2512.0	1404.0	1110.0	1404.0	3916.0	0.358529
HOUSE	5104.0	1207.0	2266.0	1207.0	6311.0	0.191253
CFOHIO	1328.0	915.0	649.0	915.0	2243.0	0.407936
CFCHICAGO	955.0	513.0	328.0	513.0	1468.0	0.349455
CFSTHEAST	987.0	475.0	361.0	475.0	1462.0	0.324897
CFVADCMD	748.0	401.0	353.0	401.0	1149.0	0.348999
CFNY	565.0	383.0	181.0	383.0	948.0	0.404008
CFSTHWEST	652.0	335.0	374.0	335.0	987.0	0.339412
CFMICHIGAN	389.0	333.0	144.0	333.0	722.0	0.461219
CFCOUTWY	385.0	239.0	144.0	239.0	624.0	0.383013

- **Insight** These salespeople have lower win rates. Consider additional training or strategy changes.

```
In [ ]: print("Bottom 10 Salespeople by Win Quantity and Win Rate:")
        print(salesperson_impact.tail(10)) # Bottom 10 salespeople sorted by win quantity and win rate
```

Bottom 10 Salespeople by Win Quantity and Win Rate:

Stage	0.0	1.0	2.0	Win_Quantity	Total_Quantity	Win_Rate
Salesperson						
CFFL	204.0	67.0	3.0	67.0	271.0	0.247232
CFNOCAL	48.0	30.0	70.0	30.0	78.0	0.384615
OPEN	15.0	26.0	2.0	26.0	41.0	0.634146
CANADAHG	132.0	21.0	27.0	21.0	153.0	0.137255
DNCFSNFRAN	21.0	17.0	2.0	17.0	38.0	0.447368
DNCFBOSTON	9.0	7.0	37.0	7.0	16.0	0.437500
Unknown	0.0	1.0	0.0	1.0	1.0	1.000000
OPALAUTRY	1.0	1.0	0.0	1.0	2.0	0.500000
DNCFNWYORK	2.0	0.0	0.0	0.0	2.0	0.000000
OPRAMOS	1.0	0.0	0.0	0.0	1.0	0.000000

5. CX AMs (Internal Account Managers)

- **Question:** How do different account managers perform in terms of win rates?
- **Analysis:**
 - **Insight:** These account managers have perfect win rates. Their methods should be analyzed for best practices.

```
In [ ]: def analyze_account_manager_impact(data, feature_name='Internal Account Manager', target_name='Stage'):
        am_success = data.groupby([feature_name, target_name]).size().unstack().fillna(0)
        am_success['Win_Quantity'] = am_success[1.0]
        am_success['Total_Quantity'] = am_success[1.0] + am_success[0.0]
        am_success['Win_Rate'] = am_success['Win_Quantity'] / am_success['Total_Quantity']
        return am_success.sort_values(by=['Win_Quantity', 'Win_Rate'], ascending=[False, False])

        account_manager_impact = analyze_account_manager_impact(data_dropped)
        print("Top 10 Account Managers by Win Quantity and Win Rate:")
        print(account_manager_impact.head(10)) # Top 10 account managers sorted by win quantity and win rate
```

Top 10 Account Managers by Win Quantity and Win Rate:

Stage	0.0	1.0	2.0	Win_Quantity \
Internal Account Manager				
Unknown	3909.0	1483.0	1391.0	1483.0
Amanda Heller	2293.0	1400.0	946.0	1400.0
Andrew Karpuk	2524.0	1121.0	1108.0	1121.0
Angela Marcotte	1779.0	1015.0	488.0	1015.0
Margie Pfriem	1713.0	807.0	858.0	807.0
Jill Hensley	853.0	653.0	604.0	653.0
Kim Johnson	2568.0	511.0	426.0	511.0
Christina Cavallo	553.0	326.0	309.0	326.0
Kristie Arrowood	334.0	157.0	306.0	157.0
Kathleen Kreacic	175.0	103.0	172.0	103.0

Stage	Total_Quantity	Win_Rate
Internal Account Manager		
Unknown	5392.0	0.275037
Amanda Heller	3693.0	0.379096
Andrew Karpuk	3645.0	0.307545
Angela Marcotte	2794.0	0.363278
Margie Pfriem	2520.0	0.320238
Jill Hensley	1506.0	0.433599
Kim Johnson	3079.0	0.165963
Christina Cavallo	879.0	0.370876
Kristie Arrowood	491.0	0.319756
Kathleen Kreacic	278.0	0.370504

- **Insight:** These account managers have lower win rates. Further training or support might be needed.

```
In [ ]: print("Bottom 10 Account Managers by Win Quantity and Win Rate:")
        print(account_manager_impact.tail(10)) # Bottom 10 account managers sorted by win quantity and win rate
```

```
Bottom 10 Account Managers by Win Quantity and Win Rate:
Stage      0.0    1.0    2.0  Win_Quantity  Total_Quantity  \
Internal Account Manager
Kathleen Kreacic      175.0  103.0  172.0         103.0         278.0
Felipe Gil             0.0   41.0  309.0         41.0         41.0
GHENT\KATHLEENK       35.0   7.0   6.0          7.0         42.0
GHENT\FILIEPEG        0.0   5.0  10.0          5.0          5.0
GHENT\JANINEK         3.0   4.0   0.0          4.0          7.0
ANDREW KARPUK         22.0   0.0   0.0          0.0         22.0
Drew Karpuk           3.0   0.0   0.0          0.0          3.0
GHENT\CORYB           1.0   0.0   0.0          0.0          1.0
GHENT\DPUCKETT        0.0   0.0   1.0          0.0          0.0
GHENT\WCSPRODBUILDER  0.0   0.0   2.0          0.0          0.0
```

```
Stage      Win_Rate
Internal Account Manager
Kathleen Kreacic      0.370504
Felipe Gil            1.000000
GHENT\KATHLEENK       0.166667
GHENT\FILIEPEG        1.000000
GHENT\JANINEK         0.571429
ANDREW KARPUK         0.000000
Drew Karpuk           0.000000
GHENT\CORYB           0.000000
GHENT\DPUCKETT        NaN
GHENT\WCSPRODBUILDER  NaN
```

6. Quote Amounts

- **Question:** How do different quote amounts (e.g., over 20,000 or under 2,000) affect win rates?
- **Analysis:**
 - **Insight:** Smaller quote amounts (0 to 2000) have the highest win rates, followed by mid-range (2000 to 20000) and high-value quotes (\$20000+). This suggests that lower-value quotes are more likely to succeed. Adjust pricing and sales strategies accordingly.

```
In [ ]: def analyze_quote_amount_impact(data, feature_name='Total Value', target_name='Stage', bins=[0, 2000, 20000, np.inf]):
        data['Value_Bin'] = pd.cut(data[feature_name], bins)
        value_bin_success = data.groupby(['Value_Bin', target_name]).size().unstack().fillna(0)
        value_bin_success['Win_Quantity'] = value_bin_success[1.0]
        value_bin_success['Total_Quantity'] = value_bin_success[1.0] + value_bin_success[0.0]
        value_bin_success['Win_Rate'] = value_bin_success['Win_Quantity'] / value_bin_success['Total_Quantity']
        return value_bin_success.sort_values(by=['Win_Quantity', 'Win_Rate'], ascending=[False, False])

quote_amount_impact = analyze_quote_amount_impact(data_dropped)
print("Top Quote Amount Bins by Win Quantity and Win Rate:")
print(quote_amount_impact)
```

```
Top Quote Amount Bins by Win Quantity and Win Rate:
Stage      0.0    1.0    2.0  Win_Quantity  Total_Quantity  Win_Rate
Value_Bin
(0.0, 2000.0]      10406  5494  4126         5494         15900  0.345535
(2000.0, 20000.0]  5341  1929  2331         1929         7270  0.265337
(20000.0, inf]     797   128   341          128          925  0.138378
```

For each of the analyses, visual representations to enhance understanding.

```
In [ ]: import seaborn as sns
        import matplotlib.pyplot as plt

        # Update plot_impact function to include annotations
        def plot_impact(data, feature_name, title):
            plt.figure(figsize=(12, 6))
            sns.barplot(x=data.index, y='Win_Rate', data=data)
            plt.title(f'Impact of {title} on Win Rate')
            plt.xlabel(feature_name)
            plt.ylabel('Win Rate')
            plt.xticks(rotation=45)
            plt.grid(True)
```

```

# Annotate each bar with the win quantity
for i, row in enumerate(data.itertuples()):
    plt.text(i, row.Win_Rate, f'{row.Win_Quantity:.0f}', color='black', ha="center", va="bottom")

plt.show()

# Sample data for illustration purposes
sample_data = pd.DataFrame({
    'Item': ['Item1', 'Item2', 'Item3', 'Item4', 'Item5'],
    'Win_Rate': [0.8, 0.6, 0.9, 0.75, 0.5],
    'Win_Quantity': [80, 60, 90, 75, 50]
})
sample_data.set_index('Item', inplace=True)

# Plotting the impact of Items on Win Rate
plot_impact(item_impact.head(10), 'Item', 'Top 10 Items')
plot_impact(customer_impact.head(10), 'Customers', 'Top 10 Customers')
plot_impact(rep_group_impact.head(10), 'Rep Groups', 'Top 10 Rep Groups')
plot_impact(salesperson_impact.head(10), 'Salespeople', 'Top 10 SalesPeople')
plot_impact(account_manager_impact.head(10), 'Account Managers', 'Top 10 Account Managers')
plot_impact(quote_amount_impact.head(10), 'Quote Amounts', 'Top 10 Quote Amounts')

```

